

Product Inventory Tracker

Using VS Code · Python Class · 2 Methods · User Input · F-String

Python class

2 methods

user input

f-string

VS Code

1. Scenario

StockSmart Inc. — Product Inventory System

You are a data analyst at StockSmart Inc. Your manager wants a Python script that models a product using a class, lets a warehouse associate enter two products via `input()`, and displays a formatted report for each product using two methods — following the same pattern as the Student class example.

- Define a class called `Product` with a constructor `__init__` that stores: name, category, price, and quantity.
- Write a method called `display_info(self)` that prints all four attributes using f-strings.
- Write a second method called `inventory_value(self)` that calculates and prints price × quantity.
- Use `input()` and type casting to collect data for two products from the user.
- Create two `Product` objects, print each object's attributes directly using f-strings, then call both methods on each object.

VS Code setup

1. Create a new file: `product_tracker.py`
2. Open the terminal: `Ctrl+`` (Win/Linux) or `Cmd+`` (Mac)
3. Run: `python product_tracker.py`

2. Requirements

Req 1 Class definition

Use the class keyword followed by the class name in PascalCase: `class Product:`. Everything indented below belongs to the class — the constructor and both methods.

Req 2 Constructor __init__	def __init__(self, name, category, price, quantity): — self is always the first parameter; it refers to the object being created. Assign each to an instance attribute: self.name = name, self.category = category, self.price = price, self.quantity = quantity.
Req 3 Method 1 — display_info	def display_info(self): — prints name, category, price (formatted :.2f), and quantity using f-strings that reference self.name, self.category, self.price, and self.quantity. Add a separator line at the end.
Req 4 Method 2 — inventory_value	def inventory_value(self): — calculates total = self.price * self.quantity inside the method body and prints the result using an f-string. Format total with :.2f.
Req 5 User input + type cast	Collect name and category as plain strings. Wrap float() around the price input and int() around quantity — input() always returns a string and the multiplication in inventory_value() requires numbers.
Req 6 2 objects + f- string display	Create product1 = Product(name1, category1, price1, quantity1) and product2 similarly. Then print each object's name and category using f-strings (just like the Student example lines 22-23), then call both methods on each object.

3. Hints

Class anatomy — Student (given) vs Product (your task)

Student class (given example)	Product class (your task)
<pre>class Student: def __init__(self, name, grade): self.name = name self.grade = grade def display_info(self): print(f"Name: {self.name}") print(f"Grade: {self.grade}") print("---")</pre>	<pre>class Product: def __init__(self, name, category, price, quantity): self.name = name self.category = category self.price = price self.quantity = quantity def display_info(self): ... def inventory_value(self): ...</pre>

1

self explained

self is how a method refers to the object it belongs to. When you write product1.display_info(), Python automatically passes product1 as self. You always declare it as the first parameter but never pass it explicitly when calling the method.

2

Two methods, same self pattern

Both methods start with `def method_name(self):`. Inside, access attributes via `self.name`, `self.price`, etc. The second method adds a calculation: `total = self.price * self.quantity` before printing.

3

Type casting for numbers

`price = float(input('Price $: '))` and `quantity = int(input('Quantity: '))`. Without the cast, Python raises a `TypeError` when `inventory_value()` tries to multiply the two values.

4

f-string currency format

Use `:.2f` inside curly braces for price and total: `f'Price: ${self.price:.2f}'` prints `Price: $29.99`. The `.2f` rounds to two decimal places.

5

Calling methods on an object

After creating `product1 = Product(...)`, call each method with `product1.display_info()` and `product1.inventory_value()`. Python injects `product1` as `self` automatically — no argument needed.

4. Solution

Attempt the exercise before reading the solution. Try writing the class from memory using only the hints and the Student example above.

product_tracker.py — complete program

```
# Deodat P. Sharma
# 5/10/2026
# Define a Python class for a product inventory tracker

class Product:
    # Constructor method
    def __init__(self, name, category, price, quantity):
        self.name = name
        self.category = category
        self.price = price
        self.quantity = quantity

    # Method 1: display product information
    def display_info(self):
        print(f"Product Name : {self.name}")
        print(f"Category      : {self.category}")
        print(f"Price           : ${self.price:.2f}")
```

```

        print(f"Quantity      : {self.quantity} units")
        print("-----")

    # Method 2: calculate total inventory value
    def inventory_value(self):
        total = self.price * self.quantity
        print(f"Inventory value for {self.name}: ${total:.2f}")
        print("-----")

# Create 2 product objects using user input
print("=== Enter Product 1 ===")
name1      =      input("Product name:  ")
category1  =      input("Category:      ")
price1     = float (input("Price $:      "))
quantity1  = int   (input("Quantity:    "))

print("\n=== Enter Product 2 ===")
name2      =      input("Product name:  ")
category2  =      input("Category:      ")
price2     = float (input("Price $:      "))
quantity2  = int   (input("Quantity:    "))

product1 = Product(name1, category1, price1, quantity1)
product2 = Product(name2, category2, price2, quantity2)

# Display using f-strings and methods
print("\n===== Product 1: " + product1.name + " =====")
print(f"Product 1: {product1.name}      {product1.category}")
product1.display_info()
product1.inventory_value()

print("===== Product 2: " + product2.name + " =====")
print(f"Product 2: {product2.name}      {product2.category}")
product2.display_info()
product2.inventory_value()

```

Key decisions explained

`class Product:` — *PascalCase class name follows Python convention; same pattern as Student.*

`self` — *always the first parameter in every method; lets methods access the object's own attributes.*

`float()` / `int()` — *type cast on input so `inventory_value()` can multiply without error.*

`product1.display_info()` — *calling a method passes `product1` as `self` automatically; no argument needed.*

`:.2f` — *formats price and total to two decimal places in the f-string output.*

5. Expected output

With the following input values:

Prompt	User types
=== Enter Product 1 ===	
Product name:	Wireless Mouse
Category:	Electronics
Price \$:	29.99
Quantity:	150
=== Enter Product 2 ===	
Product name:	USB-C Cable
Category:	Accessories
Price \$:	12.49
Quantity:	300

The terminal prints:

```
===== Product 1: Wireless Mouse =====
Product 1: Wireless Mouse   Electronics
Product Name : Wireless Mouse
Category      : Electronics
Price         : $29.99
Quantity      : 150 units
-----
Inventory value for Wireless Mouse: $4498.50
-----

===== Product 2: USB-C Cable =====
Product 2: USB-C Cable   Accessories
Product Name : USB-C Cable
Category      : Accessories
Price         : $12.49
Quantity      : 300 units
-----
Inventory value for USB-C Cable: $3747.00
-----
```

Object attribute breakdown

Attribute	product1 (Wireless Mouse)	product2 (USB-C Cable)
self.name	Wireless Mouse	USB-C Cable
self.category	Electronics	Accessories
self.price	\$29.99	\$12.49

<code>self.quantity</code>	150 units	300 units
<code>inventory_value()</code>	\$4,498.50	\$3,747.00

6. Class structure quick reference

The table below maps each part of the Product class to what it does and how it compares to the Student example.

Part	Product class	Student class (example)
Class name	<code>class Product:</code>	<code>class Student:</code>
Constructor	<code>def __init__(self, name, category, price, quantity):</code>	<code>def __init__(self, name, grade):</code>
Attributes (self)	<code>self.name, self.category, self.price, self.quantity</code>	<code>self.name, self.grade</code>
Method 1	<code>def display_info(self):</code> prints 4 attributes	<code>def display_info(self):</code> prints 2 attributes
Method 2	<code>def inventory_value(self):</code> calculates price * quantity	(not in Student example)
Objects created	<code>product1 = Product(...)</code> <code>product2 = Product(...)</code>	<code>student1 = Student(...)</code> <code>student2 = Student(...)</code>

7. Extension challenges

Once your base program runs correctly, try these extensions:

4. Add a third method `apply_discount(self, pct)` that reduces `self.price` by the given percentage and prints the new price.
5. Store both `Product` objects in a list and use a `for` loop to call `display_info()` and `inventory_value()` on each one.
6. Add a class attribute `store_name = 'StockSmart'` above `__init__` and print it inside `display_info()` using `Product.store_name`.
7. Add a fourth attribute `reorder_level` to `__init__` and write a method `needs_reorder(self)` that returns `True` if quantity is at or below `reorder_level`.
8. Rewrite the two `print(f'Product N: ...')` lines using a list comprehension that builds formatted strings for both products at once.